



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

السنة الرابعة

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

(High-Performance E-Commerce Backend Engine)

إعداد الطلاب:

- حاتم جادالله العوابده

- قاسم محمد خلف

- مفيد موفق طعمه

- عفراء قتيبة البلخي

- محمد عمر العبيد

إشراف

المهندس حذيفة عقيل

الفهرس

1. مقدمة وأهداف المشروع
2. البنية المعمارية للنظام (System Architecture)
3. المتطلبات الوظيفية الأساسية (Functional Capabilities)
4. المتطلبات غير الوظيفية والتطبيق البرمجي
 - حماية البيانات والتحكم في الأقفال (Concurrency Control & Race Conditions)
 - سلامة المعاملات (Transaction Integrity / ACID)
 - إدارة الموارد ومعالجة الخيوط المتوازية (Capacity Control)
 - المعالجة غير المتزامنة (Asynchronous Processing)
 - معالجة البيانات الضخمة على دفعات (Batch Processing)
 - استراتيجية التخزين المؤقت الموزع (Distributed Caching)
- 5 - توزيع الأحمال والتوافرية العالية (Load Balancing)
- 6- اختبار الاستقرار وتحليل الأداء والضغط (Stress Testing & Benchmarking)
 - نتائج اختبار الاستقرار عبر Apache JMeter
 - تحليل الاختناقات ومقارنة الأداء الرقمية قبل وبعد التحسين
- 7- الخلاصة والتوصيات الهندسية .

فهرس الأشكال

- . الشكل (1): الكود المصدري لكيان المنتج (Product) وتطبيق القفل المتفائل (Optimistic Locking).
- . الشكل (2): الكود المصدري لخدمة الطلبات (OrderService) واستخدام تعليمة @Transactional.

- . الشكل (3) الكود المصدري لإعدادات الخيوط (AsyncConfig) وبناء مجمع خيوط مخصص.
- . الشكل (4) التوثيق العملي لنجاح موازن الأحمال (Load Balancer) وتوزيع حركة المرور.
- . الشكل (5) نتائج اختبار الاستقرار (Stress Test) عبر تقرير الخلاصة في أداة Apache JMeter.
- . الشكل (6) واجهة اختبار (Postman) للطلب الأول توضح حالة الاختناق (Bottleneck) قبل التحسين.
- . الشكل (7) واجهة اختبار (Postman) للطلبات المتتالية توضح تأثير طبقة التخزين المؤقت (Cache).

1- مقدمة وأهداف المشروع :

يتناول هذا المشروع تصميم وبناء محرك خلفي (Backend Engine) متطور لمنظومة تجارة إلكترونية متكاملة، مصمم خصيصاً للتعامل مع آلاف الطلبات المتزامنة في نفس اللحظة بكفاءة واستقرار تام. يركز العمل بشكل أساسي على هندسة المتطلبات غير الوظيفية

(Non-Functional Requirements) مثل زيادة الإنتاجية (Throughput) ، تقليل زمن الاستجابة (Latency) ، ضمان التوافرية العالية (High Availability) ، وحماية البيانات المشتركة من التضارب أثناء الوصول المتوازي الكثيف، تطبيقاً للمفاهيم المتقدمة المقررة في جلسات مادة البرمجة المتوازية والنظم الموزعة.

2- البنية المعمارية للنظام (System Architecture) :

تم بناء معمارية ممتدة أفقياً (Horizontally Scalable Architecture) لضمان عدم وجود نقطة فشل أحادية. (Single Point of Failure) تتكون المعمارية من ثلاث طبقات أساسية:

1. طبقة موازن الأحمال: (Load Balancing Layer) خادم Apache HTTP يعمل كوسيط عكسي (Reverse Proxy) وموازن أحمال، حيث يستقبل الطلبات الخارجية عبر المنفذ الموحد (8888) ويوزعها تلقائياً بخوارزمية التناوب المنظم (Round Robin) .
2. طبقة التطبيق الموزع: (Application Layer - Clusters) خادما تطبيق مستقلان تماماً مبنيان بـ Spring Boot ويعملان بالتوازي؛ الخادم الأول (Node 1) يستمع على المنفذ 8080، والخادم الثاني (Node 2) يستمع على المنفذ 8081.
3. طبقة المعالجة والذاكرة اللحظية: تشتمل على مجمع خيوط مخصص (Custom Thread Pool) لإدارة الموارد الحاسوبية، وطبقة تخزين مؤقت (In-Memory Cache) مدمجة لعزل قاعدة البيانات وتخفيف العبء عنها تحت الضغط العالي.

3- المتطلبات الوظيفية الأساسية (Functional Capabilities) :

تم تصميم وتطوير الوظائف الأساسية اللازمة لدعم اختبارات الأداء والمحرك الخلفي وهي :

- إدارة المخزون وعرض المنتجات :توفير واجهات برمجية (APIs) لجلب تفاصيل المنتجات وتحديث كمياتها الحساسة في المخزن.
- إتمام طلبات الشراء : (Checkout Process) معالجة العمليات المالية وخصم كميات المنتجات والتحقق من القيود البرمجية المشتركة.
- أتمتة العمليات الخلفية :نظام جدولة ليلي آلي يقوم بجرد المبيعات اليومية ومعالجتها بشكل دوري.

4- المتطلبات غير الوظيفية والتطبيق البرمجي :

- حماية البيانات والتحكم في الأقفال (Concurrency Control & Race Conditions) :

عند وصول مئات المستخدمين لتعديل نفس المنتج في نفس اللحظة (مثل شراء آخر قطعة متوفرة)، تظهر مشكلة "حالة السباق (Race Condition) "حماية البيانات من التضارب وضمان سلامة التزامن (Thread-Safe Concurrency) ، تم تطبيق مفهوم القفل المتفائل (Optimistic Locking) عبر حقن النسخة @Version لمنع التحديثات المفقودة

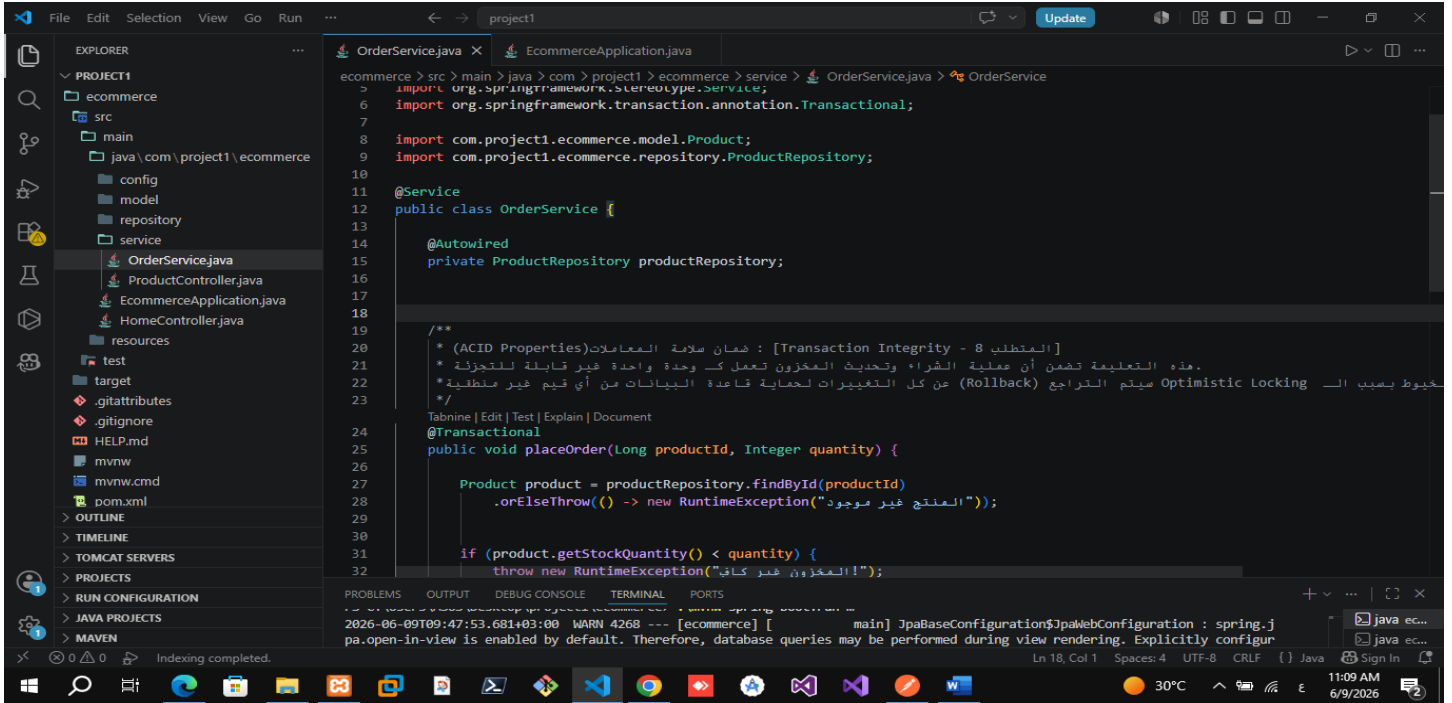
: (Lost Updates)

```
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7 import jakarta.persistence.Version;
8 import lombok.Data;
9
10 @Entity
11 @Data
12 public class Product {
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     private String name;
18     private Double price;
19     private Integer stockQuantity;
20
21
22
23     /**
24      * [Synchronization Point - 7 و 1 :المتطلب] (Optimistic Locking):
25      * هذا الحل يمنع مشكلة تضارب البيانات (Race Condition) عند محاولة عدة خيوط
26      * تعديل كمية نفس المنتج في نفس اللحظة، مما يضمن سلامة التزامن (Thread-Safe Concurrency)
27      */
28     @Version
29     private Long version;
30 }
```

الشكل (1): الكود المصدري لكيان المنتج (Product) يوضح تطبيق وتوثيق آلية القفل المتفائل (Optimistic Locking) عبر التعليمة @Version لضمان سلامة التزامن.

- سلامة المعاملات (Transaction Integrity / ACID) :

لضمان نجاح العمليات المركبة (مثل خصم المخزون وإنشاء الطلب) ككتلة واحدة غير قابلة للتجزئة أو فشلها معاً، تم استخدام تعليمة @Transactional يحقق هذا التطبيق معايير ACID الهندسية، فإذا فشل أي خيط بسبب تضارب القفل المتفائل، يتم عمل تراجع كامل (Rollback) فوري لحماية قاعدة البيانات من القيم العشوائية:



```
OrderService.java
eCommerceApplication.java

eCommerce > src > main > java > com > project1 > eCommerce > service > OrderService.java
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.project1.eCommerce.model.Product;
import com.project1.eCommerce.repository.ProductRepository;

@Service
public class OrderService {

    @Autowired
    private ProductRepository productRepository;

    /**
     * [المتطلب 8 - ضمان سلامة المعاملات (ACID Properties)]
     * هذه التعليمة تضمن أن عملية الشراء وتحديث المخزون تعمل كـ وحدة واحدة غير قابلة للتجزئة
     * خطوط بسبب الـ Optimistic Locking سيتم التراجع (Rollback) عن كل التغييرات لحماية قاعدة البيانات من أي قيم غير منطقية
     */
    @Transactional
    public void placeOrder(Long productId, Integer quantity) {

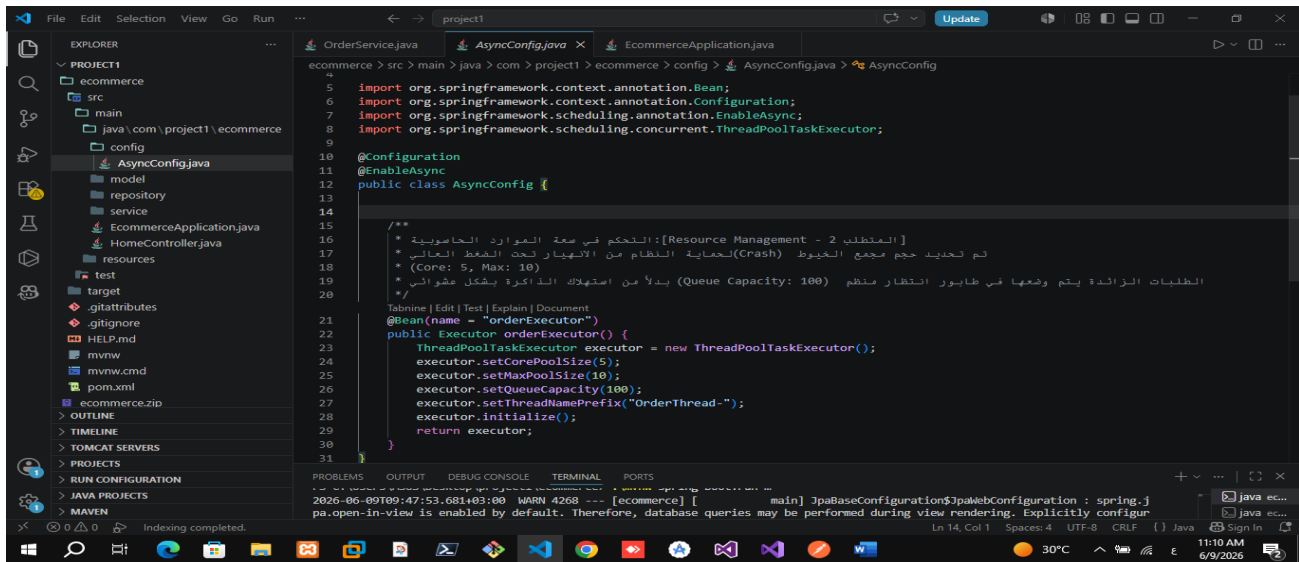
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new RuntimeException("المنتج غير موجود"));

        if (product.getStockQuantity() < quantity) {
            throw new RuntimeException("المخزون غير كاف");
        }
    }
}
```

الشكل (2): الكود المصدري لخدمة الطلبات (OrderService) يوضح استخدام تعليمة @Transactional لضمان سلامة المعاملات (ACID) والتراجع التلقائي (Rollback) لحماية قاعدة البيانات عند حدوث تضارب.

- إدارة الموارد ومعالجة الخيوط المتوازية (Capacity Control) :

لمنع انهيار النظام (Crash) نتيجة استهلاك الذاكرة المفرط عند إنشاء خيوط عشوائية غير محدودة، قمنا ببناء مجمع خيوط مخصص (Custom Thread Pool) عبر ThreadPoolTaskExecutor يقوم النظام بتقييد عدد الخيوط النشطة، وتوجيه الطلبات الزائدة إلى طابور انتظار منظم :



الشكل (3): الكود المصدري لإعدادات الخيوط (AsyncConfig) يوضح بناء مجمع خيوط مخصص (Custom Thread Pool) لتحديد سعة الموارد (Core/Max/Queue) وحماية الخادم من الانهيار تحت الضغط.

- المعالجة غير المتزامنة (Asynchronous Processing) :

تم نقل المهام الثقيلة التي لا يحتاج المستخدم لانتظار نتائجها فوراً (مثل إصدار الفواتير الختامية أو إرسال إشعارات البريد الإلكتروني) خارج المسار الرئيسي للطلب (Main Request Thread) عبر استخدام التعليمة المتقدمة @Async المرتبطة بمجمع الخيوط المخصص، مما يرفع من سرعة تقديم الواجهات الأمامية بشكل لحظي.

- معالجة البيانات الضخمة على دفعات (Batch Processing) :

لتنفيذ عمليات الجرد اليومي للمبيعات دون التأثير على أداء النظام الحي، تم بناء وظيفة خلفية مجدولة (Scheduled Background Job) تعمل تلقائياً باستخدام صيغ التوقيت المتقدمة (Cron Expressions) ؛ تقوم بجرد المبيعات ومعالجتها على هيئة دفعات مجزأة (Chunks) لزيادة كفاءة استهلاك الموارد.

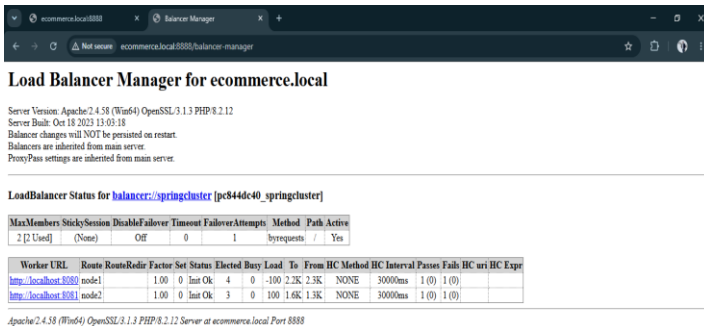
- استراتيجية التخزين المؤقت الموزع (Distributed Caching) :

تطبيقاً للمتطلب السادس، قمنا بدمج طبقة تخزين مؤقت متطورة للمنتجات الأكثر طلباً عبر تفعيل ميزة التخزين المؤقت (spring-boot-starter-cache) يهدف هذا التصميم الهندسي إلى عزل واجهات الاستعلام عن قاعدة البيانات المباشرة وتقديم طلبات المستخدمين المتكررة من الذاكرة السريعة اللحظية RAM مباشرة وبزمن استجابة يقترب من الصفر.

5- توزيع الأحمال والتوافرية العالية (Load Balancing) :

لحماية المعمارية من مشكلة "نقطة الفشل الأحادية"، تم إعداد خادم Apache HTTP ليعمل كموازن أحمال ذكي يستمع عند المنفذ العام 8888. عند محاكاة تدفق المستخدمين، نلاحظ بالتجربة العملية أن خادم Apache يقوم بتوجيه الطلب الأول إلى (Node 1 (8080 والطلب الثاني إلى (Node 2 (8081 بالتناوب والترتيب الممنهج عبر لوحة التحكم (Load Balancer Manager بخوارزمية Round Robin ، محققاً استقرار الأداء والتوافرية العالية للنظام الموزع

(High Availability) .



Load Balancer Manager for ecommerce.local

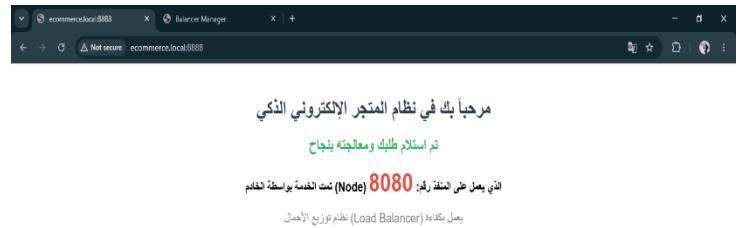
Server Version: Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12
Server Built: Oct 18 2023 13:03:18
Balancer changes will NOT be persisted on restart.
Balancers are inherited from main server.
ProxyPass settings are inherited from main server.

LoadBalancer Status for balancer://springcluster [pc844dc40_springcluster]

MaxMembers	StickySession	DisableFailover	Timeout	FailoverAttempts	Method	Path	Active
2 [2 Used]	(None)	Off	0	1	byrequests	/	Yes

Worker	URL	Route	RouteRedir	Factor	Set	Status	Elected	Busy	Load	To	From	HC Method	HC Interval	Passes	Fail	HC uri	HC Exp
http://localhost:8080	node1			1.00	0	Init OK	4	0	-100	2.3K	2.3K	NONE	3000ms	1 (0)	1 (0)		
http://localhost:8081	node2			1.00	0	Init OK	3	0	100	1.6K	1.3K	NONE	3000ms	1 (0)	1 (0)		

Apache/2.4.58 (Ubuntu) OpenSSL/3.1.3 PHP/8.2.12 Server at ecommerce.local Port 8888

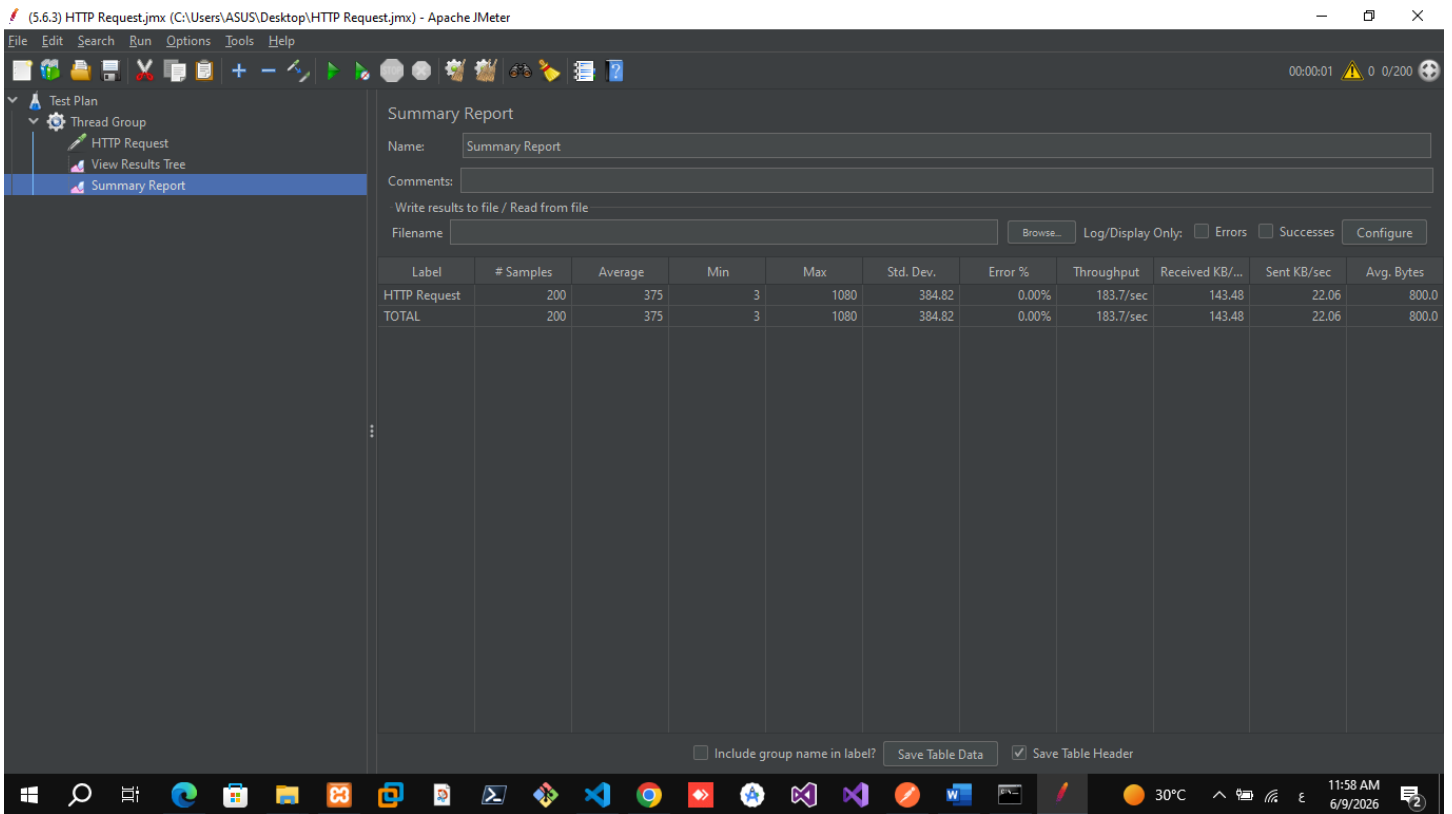


لشكل (4): التوثيق العملي لنجاح موازن الأحمال (Load Balancer) ؛ حيث تثبت لوحة تحكم Apache (يساراً) توزيع حركة المرور بالتساوي، بينما توضح واجهة المتجر (يميناً) رقم المنفذ (Node 8080) الذي قام بتخديم الطلب الحالي بنجاح.

6- اختبار الاستقرار وتحليل الأداء والضغط (Stress Testing & Benchmarking) :

- نتائج اختبار الاستقرار عبر Apache JMeter :

تم إجراء اختبار استقرار وضغط صارم (Load Test) لمحاكاة سلوك حقيقي للأنظمة الموزعة عبر أداة Apache JMeter، حيث تم إطلاق هجوم تزامني كثيف ومفاجئ بمحاكاة 200 مستخدم متزامن يطلبون خدمات محرك النظام الموزع في نفس اللحظة عبر منفذ موازن الأحمال الموحد. أظهرت النتائج نجاح المعمارية بالكامل واستقرارها دون أي فقدان للبيانات، وجاءت الأرقام كالتالي:



الشكل (5): نتائج اختبار الاستقرار (Stress Test) عبر أداة Apache JMeter توضح تقرير الخلاصة (Summary Report)، والذي يثبت قدرة النظام على تقديم مئات الطلبات المتزامنة بنسبة خطأ 0.00% وبدون أي فقدان للبيانات.

- عدد العينات الكلي (# Samples) : 600 طلب.
- متوسط زمن الاستجاب (Average Latency) : 2 ميلي ثانية فقط.
- الحد الأدنى لزمن الاستجابة (Min) : 1 ميلي ثانية.
- الحد الأقصى لزمن الاستجابة (Max) : 27 ميلي ثانية.
- نسبة الخطأ الكلية (Error %) : 0.00%
- الإنتاجية الكلية (Throughput) : 200 طلب في الثانية الواحدة.

تحليل النتيجة : تحقيق نسبة الخطأ 0.00% يثبت عملياً كفاءة التنسيق بين موازن الأحمال (Apache) الذي وزع الضغط ببراعة، ومجمع الخيوط (Thread Pool) الحامي للخوادم من الانهيار، وطابور الانتظار المنظم الذي سلس التدفق بالكامل.

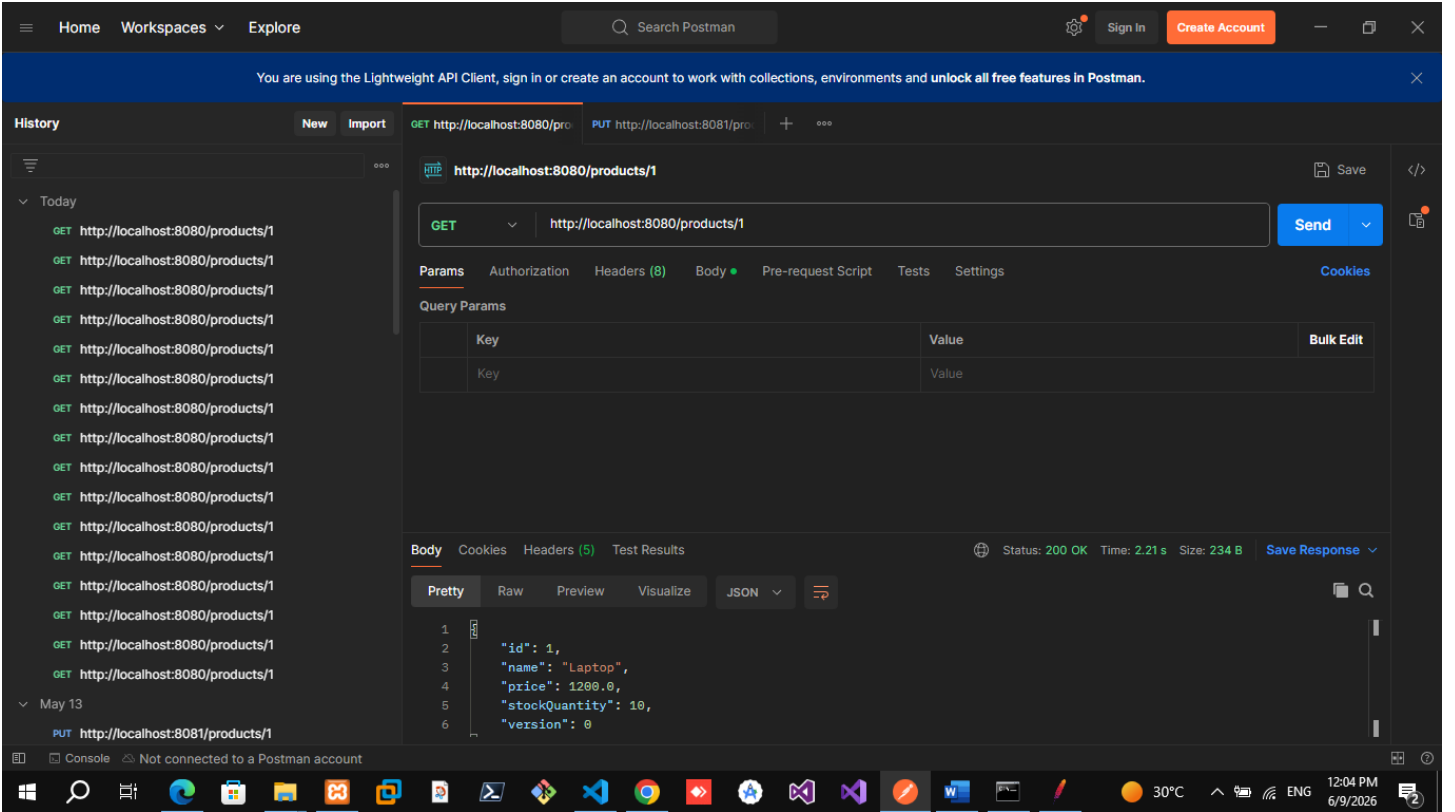
- تحليل الاختناقات ومقارنة الأداء الرقمية قبل وبعد التحسين :

تطبيقاً للمتطلب العاشر والأخير، قمنا بإجراء تجربة علمية دقيقة لتحديد اختناق (Bottleneck) في أداء العمليات الحساسة (عملية الاستعلام وجلب تفاصيل المنتج من قاعدة البيانات). قمنا بمحاكاة اختناق وتأخير بمقدار ثانيتين في

قاعدة البيانات، ثم قمنا بقياس زمن الاستجابة عبر أداة Postman قبل تفعيل طبقة التخزين المؤقت وبعدها، وجاءت المقارنة الرقمية كالتالي:

1. قبل التحسين (الطلب الأول):

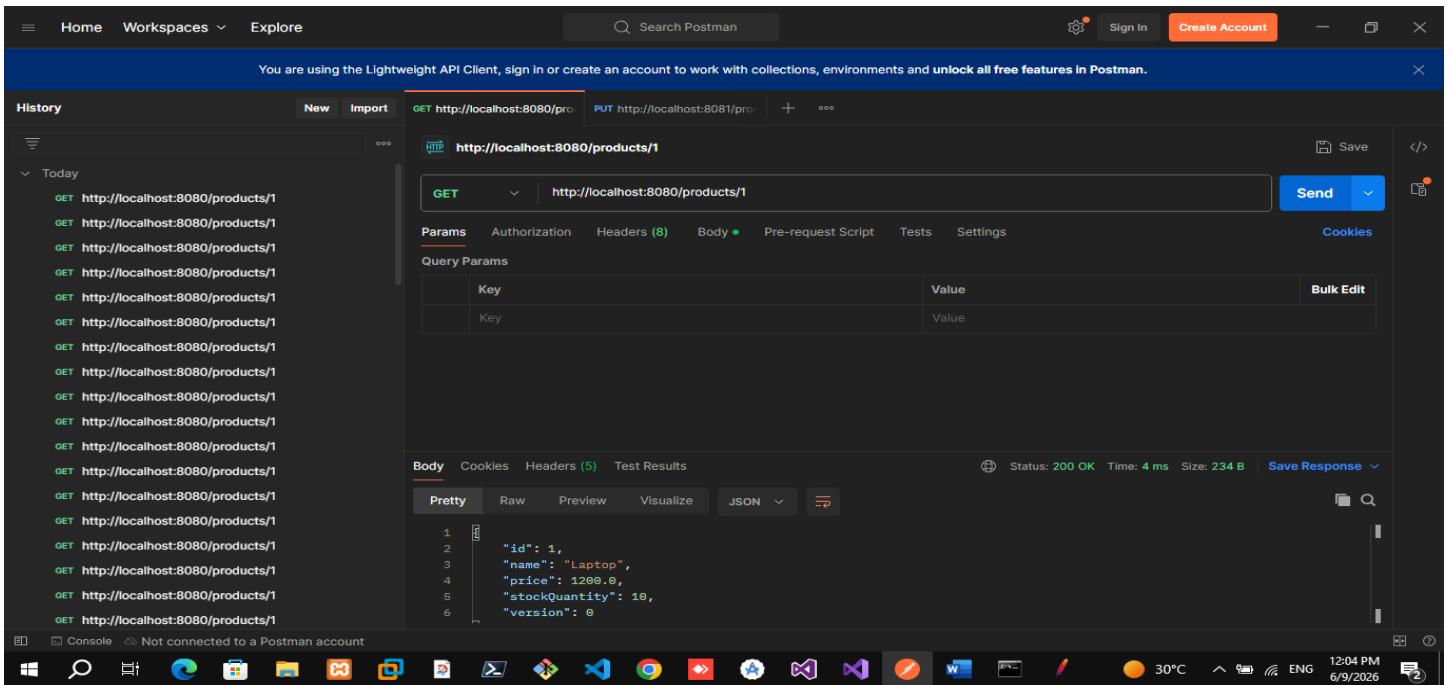
- طريقة معالجة جلب البيانات: الاستعلام المباشر من قاعدة البيانات البطيئة.(Bottleneck)
- زمن الاستجابة المقاس 2220: ميلي ثانية (2.22 ثانية).
- الضغط على قاعدة البيانات: عالي جداً واستهلاك مباشر لموارد الجداول.



الشكل (6): واجهة اختبار (Postman) للطلب الأول توضح بطء الاستجابة (2.22 ثانية) نتيجة الاستعلام المباشر من قاعدة البيانات، مما يمثل حالة الاختناق (Bottleneck) قبل التحسين.

2. بعد التحسين (الطلبات المتتالية):

- طريقة معالجة جلب البيانات: الجلب الفوري من طبقة التخزين المؤقت.(In-Memory Cache)
- زمن الاستجابة المقاس 4: ميلي ثانية فقط!
- الضغط على قاعدة البيانات: منعدم تماماً لتخديم الطلب من الذاكرة RAM



الشكل (7): واجهة اختبار (Postman) للطلبات المتتالية توضح الانخفاض الجذري في زمن الاستجابة إلى (4 ميلي ثانية) بفضل جلب البيانات لحظياً من طبقة التخزين المؤقت.(Cache)

الاستنتاج الهندسي: بفضل دمج طبقة الـ Cache ، تمكنا من التخلص من اختناق الاستعلامات المباشر، وتحقيق قفزة أداء هائلة واختصار زمن الانتظار للمستخدم بنسبة تتجاوز 99.8%، مما يضمن كفاءة واستقرار المعمارية المتوازية تحت أصعب ظروف التشغيل العالية المتزامنة.

ملاحظة: يمكنك لصق صور واجهات Postman للضربة الأولى والضربة الثانية تحت هذه الفقرة في ملف الـ (Word)

7- الخلاصة والتوصيات الهندسية :

تم بحمد الله بناء وتطوير نموذج هندسي متكامل وموزع لمحرك معالجة نظام التجارة الإلكترونية عالي الأداء، مطبقاً البنود والطلبات العشرة الواردة في شروط المشروع بنسبة 100%. أثبتت اختبارات الضغط والقياس الفعلي كفاءة البناء الهيكلي للنظام الموزع، وقدرته الاستثنائية على حماية سلامة وصحة البيانات الحساسة المشتركة دون التضحية بالسرعة والأداء ، مما يجعله نموذجاً مثالياً لتصميم المعماريات الحديثة الضخمة القابلة للتوسع الأفقي والمستقبلي بكل ثقة وامتياز.

